

# Synq.ai full product implementation plan

## Product outcome

Synq.ai becomes a managed cloud learning workspace for students and heavy note-takers. A learner signs in, submits lecture text or a supported text file, receives a validated Master Pedagogy notebook, studies it through a lesson-style Workspace, reviews concepts through quizzes and transparent mastery rules, exports learning materials, and can request an optional audio recap. The browser Workspace is the first delivery surface; the Tauri desktop client and Extension follow the core-loop quality gate rather than competing with it.

The ordinary learner never enters Gemini or ElevenLabs credentials. All provider calls remain server-side behind authenticated, quota-aware routes. Structured LectureNote-compatible data remains the source of truth; Markdown, HTML, cheat sheets, quizzes, audio scripts, and future desktop views are derived products.

## Architecture and serving decision

Use the initialized React/Vite/Express/Drizzle managed Web project as the browser application and API boundary. The browser uses CSR for the authenticated Workspace and calls same-origin server routes/tRPC procedures for source, notebook, assessment, mastery, export, and audio operations. The server owns authorization, validation, processing state, provider calls, and persistence. The managed MySQL-compatible database stores durable application records; managed object storage stores user-owned uploads, exports, and generated audio when those capabilities are enabled.

The current managed server and database stay enabled. The existing Manus OAuth/session implementation is preserved and becomes the ownership boundary for all user data. The project will not add a separate email/password system or expose provider keys to the client. Public HTML remains a small application shell; authenticated data is private and uncached. Versioned frontend assets may be long-lived after publication, while API responses and personalized HTML remain private/no-store.

## Domain model

Add additive, reviewable tables for profiles/preferences, lecture sources, processing runs, notebooks, notebook blocks or the validated canonical notebook payload, podcasts, assessments, assessment attempts, mastery items, and revisions. Every user-owned record carries an owner relationship, immutable ownership on update, timestamps, and explicit lifecycle status. The canonical LectureNote payload is stored with schema version and stable IDs so it can be re-rendered or exported without replaying a provider call. Provider

model/prompt metadata is stored in processing-run metadata, never in learner prose or secrets.

Source ingestion accepts pasted text and `.txt` / `.md` uploads first. Audio/video transcription, broad file formats, collaboration, local AI, and Extension capture remain behind explicit later gates. Uploads receive size/type validation before durable storage and are indexed in the database with soft-delete/export semantics.

## Product stages

### Stage 1 — Real core loop

Replace fixture-only intake with source creation, persisted processing states, authenticated source ownership, and a server-side generation procedure. Finalize the Master Pedagogy request envelope and validate every generated response with the LectureNote schema. Start synchronously, record duration/token metadata, and benchmark 20-, 60-, and 90-minute sources. If the 90-minute path cannot fit the configured execution ceiling including retries, move the same domain procedure behind a durable job runner rather than rewriting the contract.

The Workspace reads the stored notebook and source records, shows pending/processing/completed/failed states, supports retry without duplicate records, and preserves the source when generation fails. Provider errors are sanitized and actionable. No raw source, prompt, generated content, or provider response is logged by default.

### Stage 2 — Production Workspace

Complete the notebook experience around the approved content vocabulary: orientation, objectives, definitions, explanations, examples, comparisons, processes, formulas, misconceptions, connections, knowledge checks, summaries, revision prompts, transcript/source references, and uncertainty. Add inline annotations/basic edits where ownership and revision semantics are clear. Add real lecture library search/filtering, source detail, export to Markdown/HTML, downloadable learning materials, and sync-pending indicators.

Keep the central reading column dominant. Secondary source context, outline, audio, progress, and actions use progressive disclosure. Add semantic focus states, keyboard navigation, reduced-motion behavior, readable contrast, mobile/responsive layout, and empty/error/loading states.

### Stage 3 — Assessment and mastery

Generate and persist knowledge checks from validated notebook content. Store attempts and transparent outcomes. Start mastery with explainable rules based on correctness, recency, and review history. Surface due concepts, weak concepts, revision prompts, and a small mastery overview. Do not add opaque adaptive personalization until the transparent model is validated.

Add cheat-sheet derivation from the notebook and keep it version-linked. A notebook revision invalidates or re-generates derived assessment/cheat-sheet artifacts explicitly rather than silently mixing versions.

## **Stage 4 — Audio recap**

Generate a clean spoken script from a completed notebook, request one default language/voice through the protected server-side audio integration, store the result privately, and expose playback/download metadata. Podcast status is independent from notebook status: audio failure never invalidates a usable notebook. Cache audio per notebook version and provide retry.

## **Stage 5 — Account, privacy, and release hardening**

Finish account/profile/settings, usage/quota visibility, service status, source/notebook export, permanent deletion with deliberate cascade/soft-delete behavior, privacy copy, rate limits, abuse controls, observability without raw-content logging, and provider failure tests. Verify ownership on every query and mutation. Add golden-set evaluation artifacts and gate the first public-quality release on no factual-accuracy or faithfulness score below 3/5 and a 4.0/5 average across all five rubric dimensions.

## **Stage 6 — Desktop and Extension after the gate**

Create the Tauri client only after the browser core loop clears the golden set. Use PKCE for OAuth and Tauri Stronghold for refresh-token storage; keep access tokens memory-only. Reuse the web domain contracts. Build the Extension after the ingestion contract is stable, beginning with selected text/page context and explicit consent; defer audio capture and broad permissions.

## **Immediate implementation tranche**

1. Add the full plan and acceptance ToDo to the managed project.
2. Add durable source/processing/notebook domain types and additive database schema/migrations.
3. Add authenticated source and notebook procedures with ownership checks and tests.

4. Add the server-side Master Pedagogy adapter using the managed LLM runtime contract, strict response parsing, schema validation, sanitized errors, and processing-state transitions.
5. Replace the fixture-only Workspace reads with real API-backed source/library/notebook states while retaining the fixture as a deterministic test seed.
6. Add source/library/export UI, then assessment/mastery persistence and UI.
7. Add audio only after notebook generation and persistence are stable.

## Verification strategy

Run `pnpm check`, the full Vitest suite, targeted route/domain tests, database migration checks, and `pnpm build` after each tranche. Test ownership boundaries, duplicate-submit/idempotency behavior, invalid contracts, malformed/empty provider output, transient retry behavior, quota failures, source preservation on generation failure, and notebook rendering from stored structured data. Verify `/manus-routes.json`, health, direct routes, and the public Preview after server changes. Browser screenshots/interactions are not used as a default test method; use them only for an observed visual/interaction defect or an explicit request.

The release gate includes a fixed 15–20 item golden set spanning formula-heavy, fact-dense, interpretive, and edge-case sources. Record factual accuracy, faithfulness, pedagogical clarity, completeness, and uncertainty handling, with the approved minimums above. No desktop, Extension, collaboration, advanced mastery, local AI, or broad file-format work is considered complete before the core loop clears that gate.

## Current status and next task

The managed Web project has a polished fixture-backed Workspace, functional navigation/subpages, a LectureNote-compatible runtime schema/fixture/exporter, route manifest, branding, and passing typecheck/tests/build. The durable domain tranche is now in place: additive source, processing-run, notebook, assessment, mastery, and revision tables are defined and migrated; protected source/notebook/workspace procedures exist; and the server-side Master Pedagogy adapter validates JSON against the LectureNote contract and persists pending/processing/completed/failed transitions with duplicate protection. The next code tranche is to connect the real authenticated Workspace intake/library to these procedures, then add generated assessments/mastery and replace remaining fixture-only states incrementally.